

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: Week 4

Student Name: J.Srikar

Roll No.: A24126552083

Date: 25-08-2026

1. TITLE

Develop Dynamic Webpage Using Java Script

2. AIM

To design and develop an interactive dynamic webpage using HTML, CSS, and client-side JavaScript that demonstrates Document Object Model (DOM) manipulation, dynamic content modification, algorithmic random color styling, dynamic creation and appending of list elements with input validation, and asynchronous real-time clock rendering.

3. OBJECTIVE

- * To understand client-side DOM (Document Object Model) access and element selection via `document.getElementById()`.
- * To handle user interaction events via inline event handlers (onclick).
- * To dynamically alter text node content using the `textContent` property without refreshing the page.
- * To dynamically modify CSS styles (`document.body.style.backgroundColor`) with random hexadecimal color generation.
- * To perform client-side input validation and error feedback using browser `alert()` dialogs.
- * To instantiate, configure, and mount new DOM nodes dynamically using `document.createElement()` and `appendChild()`.
- * To implement asynchronous recurring execution using `setInterval()` and the JavaScript Date API for live time tracking.

4. THEORY

A dynamic webpage is a webpage whose content, presentation, or behavior can change in real time in response to user actions or automated programmatic events without requiring a full page reload from the web server. Client-side scripting with JavaScript is the foundational standard for delivering interactivity on the modern web.

Key structural concepts implemented in this application:

- * Document Object Model (DOM): The DOM is an object-oriented, hierarchical tree representation of an HTML document created by the browser. Each HTML tag is represented as an element node with accessible attributes, properties, and methods.
- * DOM Selection & Mutation: JavaScript provides selectors such as `document.getElementById()` to retrieve specific elements. The `.textContent` property allows reading and safe writing of element text nodes without triggering costly HTML re-parsing.
- * Dynamic CSS Manipulation: Element style properties can be accessed and overridden directly via the `element.style` property. For example, `document.body.style.backgroundColor = color` dynamically applies runtime styling changes.
- * Random Color Generation: Colors in CSS can be defined in hexadecimal format (`#RRGGBB`). Generating a random integer between 0 and 16777215 (`0xFFFFFFFF`) using `Math.floor(Math.random() * 16777215)` and converting it with `.toString(16)` provides 16.7 million possible hex color values.
- * Dynamic Element Creation: Rather than hardcoding lists, elements can be created in memory via `document.createElement('li')`, configured with content (`newItem.textContent = itemText`), and inserted into the active DOM tree using parent node's `.appendChild()`.
- * Form Input Validation: Validating inputs before processing prevents empty or invalid data submissions. If the `.value` of an input element is empty (`""`), execution is aborted, and a prompt is displayed to the user using `alert()`.
- * Asynchronous Timers (`setInterval`) & Date API: JavaScript's new `Date()` object encapsulates current date, month, year, hours, minutes, and seconds. Passing `updateDateTime` to `setInterval(..., 1000)` creates a recurring timer triggering every 1000 ms to display a live ticking clock.

5. ALGORITHM / PROCEDURE

1. Initialize the HTML5 document structure with <!DOCTYPE html>, <html>, <head>, <style>, and <body> tags.
2. Define CSS styles for page background, container card, buttons, text inputs, hover effects, and list items.
3. Construct the HTML body containing heading, message paragraph (#message), and action buttons.
4. Add an input box (<input id='itemInput'>) alongside an 'Add Item' button and an unordered list (<ul id='itemList'>).
5. Add a real-time clock container (<p id='dateTime'>) to host dynamic date and time outputs.
6. Implement changeText() to fetch #message via document.getElementById() and update.textContent to the dynamic status text.
7. Implement changeBackground() to compute Math.floor(Math.random() * 16777215).toString(16) and set document.body.style.backgroundColor.
8. Implement addItem() to validate input (alerting if empty), create with createElement(), set.textContent, append to #itemList, and clear input.
9. Implement updateDateTime() to fetch new Date(), format with toLocaleString(), and assign to #dateTime.textContent.
10. Register setInterval(updateDateTime, 1000) to tick every second, and invoke updateDateTime() once immediately for initial rendering.

6. PROGRAM

week4.html (Part 1: HTML Head, Document Setup & Internal CSS Styling)

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Dynamic Webpage Using JavaScript</title>

  <style>
    /* Page styling */
    body {
      font-family: Arial, sans-serif;
      background-color: lightblue;
      margin: 0;
      padding: 0;
    }

    /* Main container */
    .container {
      width: 80%;
      max-width: 700px;
      margin: 50px auto;
      padding: 30px;
      background-color: white;
      text-align: center;
      border-radius: 10px;
      box-shadow: 0 0 10px gray;
    }

    h1 { color: darkblue; }
    #message { font-size: 20px; color: blue; }
    input { padding: 10px; width: 60%; margin: 10px; }

    button {
      padding: 10px 15px;
      margin: 5px;
      border: none;
      border-radius: 5px;
      background-color: blue;
      color: white;
      cursor: pointer;
    }

    button:hover { background-color: darkblue; }
    ul { list-style-type: none; padding: 0; }
    li { background-color: lightgray; margin: 8px; padding: 10px; border-radius: 5px }
  </style>
</head>
```

week4.html (Part 2: HTML Body, Dynamic UI Controls & JavaScript Functions)

```
<body>
  <div class="container">
    <h1>Dynamic Webpage</h1>
    <p id="message">Welcome to my dynamic webpage!</p>

    <!-- Change text -->
    <button onClick="changeText()">Change Text</button>

    <!-- Change background -->
    <button onClick="changeBackground()">Change Background</button>
    <hr>

    <h2>Dynamic Item List</h2>
    <input type="text" id="itemInput" placeholder="Enter an item">
    <button onClick="addItem()">Add Item</button>
    <ul id="itemList"></ul>
    <hr>

    <h2>Current Date and Time</h2>
    <p id="dateTime"></p>
  </div>

  <script>
    // Function to change the paragraph text
    function changeText() {
      document.getElementById("message").textContent =
        "The webpage has been changed dynamically using JavaScript!";
    }

    // Function to change the background color
    function changeBackground() {
      // Generate a random color
      let color = "#" + Math.floor(Math.random() * 16777215).toString(16);
      // Change the background color
      document.body.style.backgroundColor = color;
    }

    // Function to add a new item
    function addItem() {
      // Get the input element
      let input = document.getElementById("itemInput");
      // Get the entered text
      let itemText = input.value;
      // Check whether input is empty
      if (itemText === "") {
        alert("Please enter an item!");
        return;
      }
      // Create a new list item
      let newItem = document.createElement("li");
      // Add text to the list item
      newItem.textContent = itemText;
      // Add the list item to the webpage
      document.getElementById("itemList").appendChild(newItem);
      // Clear the input box
      input.value = "";
    }
  </script>
</body>
```

week4.html (Part 3: Clock Timer, Finalization & Closing Tags)

```
// Function to display date and time
function updateDateTime() {
    let currentDate = new Date();
    document.getElementById("dateTime").textContent = currentDate.toLocaleString()
}

// Update date and time every second
setInterval(updateDateTime, 1000);
// Display date and time when page loads
updateDateTime();
</script>
</body>
</html>
```

7. CODE EXPLANATION

Detailed technical breakdown of the HTML elements, CSS rules, and JavaScript DOM manipulation logic:

Code / Element	Explanation
<!DOCTYPE html>, <html>, <body>	Core structural HTML5 root elements declaring standards mode and document containers.
<style> .container { ... } </style>	Internal CSS defining responsive card layout (width: 80%, max-width: 700px, box-shadow).
<button onclick="changeText()">	Interactive button binding user click events to invoke the JavaScript changeText() function.
document.getElementById("message")	DOM query method selecting the specific element node carrying the unique ID 'message'.
.textContent = "..."	Property that updates the textual node content safely without reparsing HTML tags.
Math.random() * 16777215	Generates a random numerical value within the 24-bit RGB hexadecimal color spectrum.
.toString(16)	Number prototype method converting the integer into its base-16 hexadecimal representation.
document.body.style.backgroundColor	Directly mutates the inline CSS background-color style of the document body element.
if (itemText === "") alert(...);	Input validation routine displaying a modal warning dialog when input is empty.
document.createElement("li")	Programmatically creates a new HTML list item element () in browser memory.
appendChild(newItem)	Inserts the newly created element as a child of the container in the DOM tree.
let currentDate = new Date()	Instantiates a JavaScript Date object capturing current system date and clock timestamp.
currentDate.toLocaleString()	Formats the date and time instance into a readable regional string representation.
setInterval(updateDateTime, 1000)	Schedules the updateDateTime function to execute asynchronously every 1000ms (1 second).
updateDateTime()	Initial direct function call to render the clock immediately on page load without delay.

8. TEST CASES AND EXECUTION DATA

The dynamic webpage events, DOM mutations, and script behaviors were tested and validated:

Test Case	Feature Tested	Action / Input	Expected Result	Actual Result	Status
TC-01	Page Load Layout	Load week4.html in browser	Render container card, buttons, input	Displayed cleanly	Pass
TC-02	Initial Message	Inspect #message on load	Show 'Welcome to my dynamic webpage!'	Text verified	Pass
TC-03	Change Text Event	Click 'Change Text' button	Update message text via textContent	Message updated	Pass
TC-04	Change Background	Click 'Change Background'	Apply random hex color to body style	Background changed	Pass
TC-05	Consecutive Colors	Click color button 5 times	Generates distinct hex colors each time	Colors vary randomly	Pass
TC-06	Add Valid Item	Input 'Milk' & click Add Item	Append new element to #itemList	Item added to list	Pass
TC-07	Input Box Reset	Inspect input after adding item	Input field reset to empty string "	Input box cleared	Pass
TC-08	Empty Input Alert	Click Add Item with empty input	Trigger browser alert 'Please enter an item!'	Alert dialog shown	Pass
TC-09	Clock Initial Render	Inspect #dateTime on load	Display localized current date & time	Clock displayed	Pass
TC-10	Clock Ticking Timer	Observe clock for 3 seconds	Seconds increment every 1000ms	Time ticks dynamically	Pass

Additional Boundary & Edge Case Validation

Test Case	Input / Scenario	Expected Behavioral Result	Status
TC-11	Item text containing symbols ('HTML & CSS + JS!')	Correctly creates text node without HTML injection vulnerabilities	Pass
TC-12	Item text containing numerals ('Assignment #4')	Accurately represents string data inside styled list element	Pass
TC-13	Multiple items added in sequence (5 items)	Maintains strict FIFO order and renders distinct child nodes	Pass
TC-14	Continuous timer execution (over 5 minutes)	Clock maintains accurate interval cadence without memory leaks	Pass

9. RESULT

Thus, an interactive dynamic webpage was successfully developed and verified using HTML, CSS, and client-side JavaScript. The implementation demonstrated core Document Object Model (DOM) manipulation techniques including targeted element selection, dynamic text updating, algorithmic random background color generation, dynamic element creation and appending with input validation, and real-time clock rendering using asynchronous timer intervals without any runtime errors.